K8s - Partie n°2

> **Étudiant :** David MELOCCO, TD1 BUT3 **Date :** vendredi 6 novembre 2026

Prérequis

L'ensemble des travaux pratiques sont faits sous **Windows** et **PowerShell**. Une configuration **WSL2** est déjà installée pour **Docker Desktop**. L'extension **VS Code Kubernetes**, proposée par **Microsoft**, a été installée.

> 📦 Le dépôt **GitHub** est open-source, disponible ici : https://github.com/ThFoxY/kubernetes-e-learn-lab

MySQL

1. 📄 Créer un fichier YAML `mysql-secret.yml`, ajoutez-y une ressource de type `Secret` avec ces différentes clé/valeur :

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
type: Opaque
data:
  MYSQL_PASSWORD: <mot_de_passe>
  MYSQL_ROOT_PASSWORD: <mot_de_passe>
```

> **Le dépôt est public, et je souhaite le garde ainsi, afin d'avoir une preuve d'un lab Kubernetes sur mon profil GitHub.**

`<mot_de_passe>` a été remplacé par un mot de passe encodé en **base64**, et le fichier a été **.gitignored**.

---

2. 📄 Créer un fichier YAML `mysql-configmap.yml`, ajoutez-y une ressource de type `ConfigMap` avec ces différentes clé/valeur :

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: mysql-configmap
data:
  MYSQL_USER: userdb
  MYSQL_DATABASE: wordpress
```

---

3. 📄 Créer un fichier YAML nommé `mysql-statefulset.yml`. Dans ce fichier, définissez une ressource de type `StatefulSet` avec :

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql-statefulset
spec:
  serviceName: mysql
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:9.0
          env:
            - name: MYSQL_USER
              valueFrom:
                configMapKeyRef:
                  name: mysql-configmap
                  key: MYSQL_USER
            - name: MYSQL_DATABASE
```

```
              valueFrom:
                configMapKeyRef:
                  name: mysql-configmap
                  key: MYSQL_DATABASE
            - name: MYSQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_PASSWORD
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_ROOT_PASSWORD
          ports:
            - containerPort: 3306
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
  volumeClaimTemplates:
    - metadata:
        name: mysql-persistent-storage
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 100Mi
```

---

4. 📄 Créer un fichier YAML nommé `mysql-service.yml`. Dans ce fichier, définissez une ressource de type `Service` qui écoutera sur le port `3306` et redirigera les requêtes vers le port `3306` des pods ayant le label `app: mysql`

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-svc
```

```yaml
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
```

---

5. 📸 Connexion au pod MySQL et vérifier la présence des 4 variables d'environnement avec les valeurs définies dans les ressources ConfigMap et Secret

```
kubectl apply -f .\database\

configmap/mysql-configmap created
secret/mysql-secret created
service/mysql-svc created
statefulset.apps/mysql-statefulset created
namespace/database unchanged

kubectl get pods

NAME                 READY   STATUS    RESTARTS   AGE
mysql-statefulset-0   1/1     Running   0          2m
```

```
david ~ 0ms kubectl get pods
NAME                  READY   STATUS    RESTARTS   AGE
mysql-statefulset-0   1/1     Running   0          6m52s
david ~ 256ms kubectl exec -it mysql-statefulset-0 -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=mysql-statefulset-0
GOSU_VERSION=1.17
MYSQL_MAJOR=innovation
MYSQL_VERSION=9.0.1-1.el9
MYSQL_SHELL_VERSION=9.0.1-1.el9
MYSQL_DATABASE=wordpress
MYSQL_PASSWORD=MotDePasseSecurise1234
MYSQL_ROOT_PASSWORD=MotDePasseSecurise4321
MYSQL_USER=userdb
KUBERNETES_PORT_443_TCP_PORT=443
MYSQL_SVC_SERVICE_PORT=3306
MYSQL_SVC_PORT=tcp://10.101.78.41:3306
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_PORT_HTTPS=443
```

`MYSQL_DATABASE`, `MYSQL_PASSWORD`, `MYSQL_ROOT_PASSWORD` et `MYSQL_USER` y sont bien.

WordPress

1. 📄 Créer un fichier YAML `wordpress-pv.yml`. Dans ce fichier, ajoutez-y une ressource de type `PersistentVolume` qui a pour nom `wordpress-pv`. Ce volume persistant doit avoir une capacité de `100Mi`, un mode d'accès en `lecture/écriture` pour `l'ensemble des noeuds` du cluster et les données seront stockées dans `/tmp/kubernetes/wordpress`

Premièrement, j'ai décidé de corriger `1024Mi` pour `100Mi` que j'avais mis par convention dans `mysql-statefulset.yml`. Je l'ai donc corrigé ainsi :

```
kubectl apply -f .\database\

configmap/mysql-configmap unchanged
secret/mysql-secret unchanged
service/mysql-svc unchanged
namespace/database unchanged
The StatefulSet "mysql-statefulset" is invalid: spec.volume
ClaimTemplates: Invalid value: [{"name":"mysql-persistent-s
torage","Spec":{"AccessModes":["ReadWriteOnce"],"Selector":
```

```
null,"Resources":{"Limits":null,"Requests":{"storage":"100M
i"}},"VolumeName":"","StorageClassName":null,"VolumeMod
e":"Filesystem","DataSource":null,"DataSourceRef":null,"Vol
umeAttributesClassName":null},"Status":{"Phase":"Pendin
g","AccessModes":null,"Capacity":null,"Conditions":null,"Al
locatedResources":null,"AllocatedResourceStatuses":null,"Cu
rrentVolumeAttributesClassName":null,"ModifyVolumeStatus":n
ull,"HealthStatus":null}}]: field is immutable
```

> `field is immutable`, **car j'ai modifié un champ qui ne doit pas être modifié.**

Selon https://kubernetes.io/docs/tasks/run-application/delete-stateful-set/, la commande est la suivante :

```
kubectl delete statefulsets mysql-statefulset

statefulset.apps "mysql-statefulset" deleted from default n
amespace
```

En relançant la commande, j'obtiens la création :

```
kubectl apply -f .\database\

configmap/mysql-configmap unchanged
secret/mysql-secret unchanged
service/mysql-svc unchanged
statefulset.apps/mysql-statefulset created
namespace/database unchanged
```

Puis, j'ai continué la configuration **YAML** du **WordPress** :

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: wordpress-pv
spec:
  capacity:
    storage: 100Mi
```

```
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Delete
  storageClassName: manual
  hostPath:
    path: /tmp/kubernetes/wordpress
```

---

2. 📄 Créer un fichier YAML `wordpress-pvc.yml`. Dans ce fichier, ajoutez-y une ressource de type `PersistentVolumeClaim` qui a pour nom `wordpress-pvc`. Cette demande de stockage a pour exigences : une capacité de `100Mi` et un mode d'accès en `lecture/écriture` pour `l'ensemble des noeuds` du cluster

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: wordpress-pvc
spec:
  resources:
    requests:
      storage: 100Mi
  accessModes:
    - ReadWriteMany
  storageClassName: manual
```

---

3. Déployer la ressource Secret `mysql-secret` au sein du namespace `middle`

```
kubectl apply -f .\database\mysql-secret.yml -n middle

secret/mysql-secret created
```

---

4. Déployer la ressource Secret `mysql-configmap` au sein du namespace `middle`

```
kubectl apply -f .\database\mysql-configmap.yml -n middle

configmap/mysql-configmap created
```

---

5. 📄 Créer un fichier YAML nommé `wordpress-deployment.yml`. Dans ce fichier, définissez une ressource de type `Deployment`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: blog
  template:
    metadata:
      labels:
        app: blog
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
          env:
            - name: WORDPRESS_DB_HOST
              value: mysql-svc.database.svc.cluster.local
            - name: WORDPRESS_DB_USER
              valueFrom:
                configMapKeyRef:
                  name: mysql-configmap
                  key: MYSQL_USER
            - name: WORDPRESS_DB_NAME
              valueFrom:
                configMapKeyRef:
```

```
                    name: mysql-configmap
                    key: MYSQL_DATABASE
              - name: WORDPRESS_DB_PASSWORD
                valueFrom:
                  secretKeyRef:
                    name: mysql-secret
                    key: MYSQL_PASSWORD
              - name: WORDPRESS_CONFIG_EXTRA
                value: |
                  define('WP_HOME', 'http://localhost:8080');
                  define('WP_SITEURL', 'http://localhost:808
0');
          volumes:
            - name: wordpress-persistent-storage
              persistentVolumeClaim:
                claimName: wordpress-pvc
```

L'extension VS Code Kubernetes indique une mauvaise pratique de code : il faut inclure des `requests` et `limits` dans `resources` ("One or more containers do not have resources - this can cause noisy neighbor issues").

Entre temps, j'en ai profité pour corriger `port: 8383` dans `nginx-service.yml` pour :

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: reverse-proxy
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 80
```

---

6. 📄 Créer un fichier YAML nommé `wordpress-service.yml`. Dans ce fichier, définissez une ressource de type `Service` qui

écoutera sur le port `80` et redirigera les requêtes vers le port `80` des pods ayant le label `app: blog`

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-svc
spec:
  selector:
    app: blog
  ports:
    - port: 80
      targetPort: 80
```

---

7. 📸 Connexion au pod Wordpress et vérifier la présence des 4 variables d'environnement avec les valeurs définies

Je déploie rapidement :

```
kubectl apply -f .\middle\

namespace/middle unchanged
persistentvolume/wordpress-pv created
persistentvolumeclaim/wordpress-pvc created
service/wordpress-svc created
Error from server (BadRequest): error when creating "middle
\\wordpress-deployment.yml": Deployment in version "v1" can
not be handled as a Deployment: strict decoding error: unkn
own field "spec.template.spec.containers[0].volumes"
```

> **`create` existe, mais `apply` évite la redondance si quelque chose existe déjà.**

Il y a une erreur à la création du déploiement **WordPress**, car le fichier `wordpress-deployment.yml` était mal indenté. `volumes` doit être au même niveau d'indentation que `containers` :

```
apiVersion: apps/v1
kind: Deployment
```

```yaml
metadata:
  name: wordpress-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: blog
  template:
    metadata:
      labels:
        app: blog
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
          env:
            - name: WORDPRESS_DB_HOST
              value: mysql-svc.database.svc.cluster.local
            - name: WORDPRESS_DB_USER
              valueFrom:
                configMapKeyRef:
                  name: mysql-configmap
                  key: MYSQL_USER
            - name: WORDPRESS_DB_NAME
              valueFrom:
                configMapKeyRef:
                  name: mysql-configmap
                  key: MYSQL_DATABASE
            - name: WORDPRESS_DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_PASSWORD
            - name: WORDPRESS_CONFIG_EXTRA
              value: |
                define('WP_HOME', 'http://localhost:8080');
```

```
                define('WP_SITEURL', 'http://localhost:808
0');
      volumes:
        - name: wordpress-persistent-storage
          persistentVolumeClaim:
            claimName: wordpress-pvc
```

Et je redéploie ce fichier uniquement :

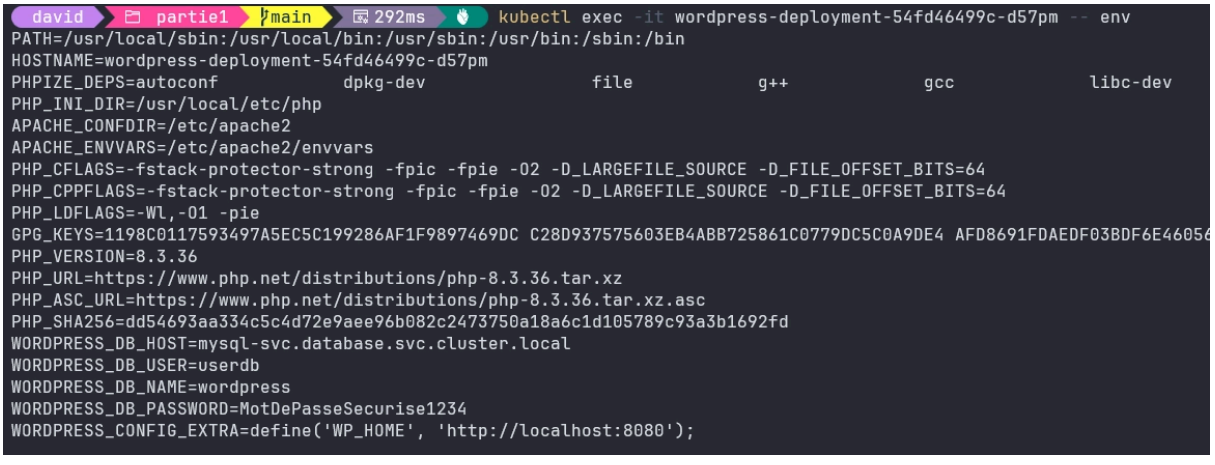
```
kubectl apply -f .\middle\wordpress-deployment.yml

deployment.apps/wordpress-deployment created
```

Désormais, le pod **WordPress** est **RUNNING** :

```
kubectl get pods

NAME                                READY   STATUS    R
ESTARTS   AGE
mysql-statefulset-0                 1/1     Running   0
41m
wordpress-deployment-54fd46499c-d57pm   1/1     Running   0
3m28s
```



Nginx

1. 📄 Créer un fichier YAML `nginx-cm.yml`, au sein de ce fichier ajoutez-y une ressource de type `ConfigMap` qui a pour clé-valeur le fichier de configuration `default.conf` correctement parametré pour rediriger les requetes vers le pod Wordpress

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-cm
data:
  default.conf: |
    server {
        listen       80;
        server_name  localhost;
        location / {
            proxy_pass         http://wordpress-svc.middle.
svc.cluster.local;
            proxy_set_header   Host $host;
            proxy_set_header   X-Real-IP $remote_addr;
            proxy_set_header   X-Forwarded-For $proxy_add_x
_forwarded_for;
            proxy_set_header   X-Forwarded-Proto $scheme;
        }
        access_log /var/log/nginx/access.log;
        error_log  /var/log/nginx/error.log;
    }
```

---

2. Créer une redirection de port vers le service Wordpress

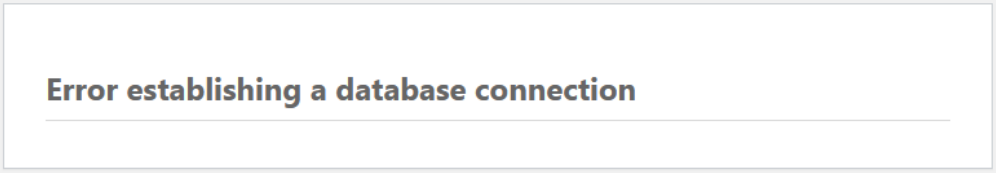
```
kubectl port-forward service/wordpress-svc 8080:80

Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Je remarque qu'ajouter `-n middle` ne fonctionne plus : j'ai oublié de spécifier un déploiement dans le namespace `middle`. En vérifiant, ils sont tous dans `default`.

---

3. 📸 Accéder à la page d'accueil de votre site depuis votre navigateur



Il y a une erreur de connexion à la base de données. Il me semble que c'est dû au namespacing.

```
kubectl get pods -n front

nginx-deployment-5d68b47d97-s8p4r   1/1     Running   0
3h36m
nginx-deployment-5d68b47d97-vs5bp   1/1     Running   0
3h36m
```

Deux déploiements dans `front`, mais :

```
kubectl get pods -n database && kubectl get pods -n middle

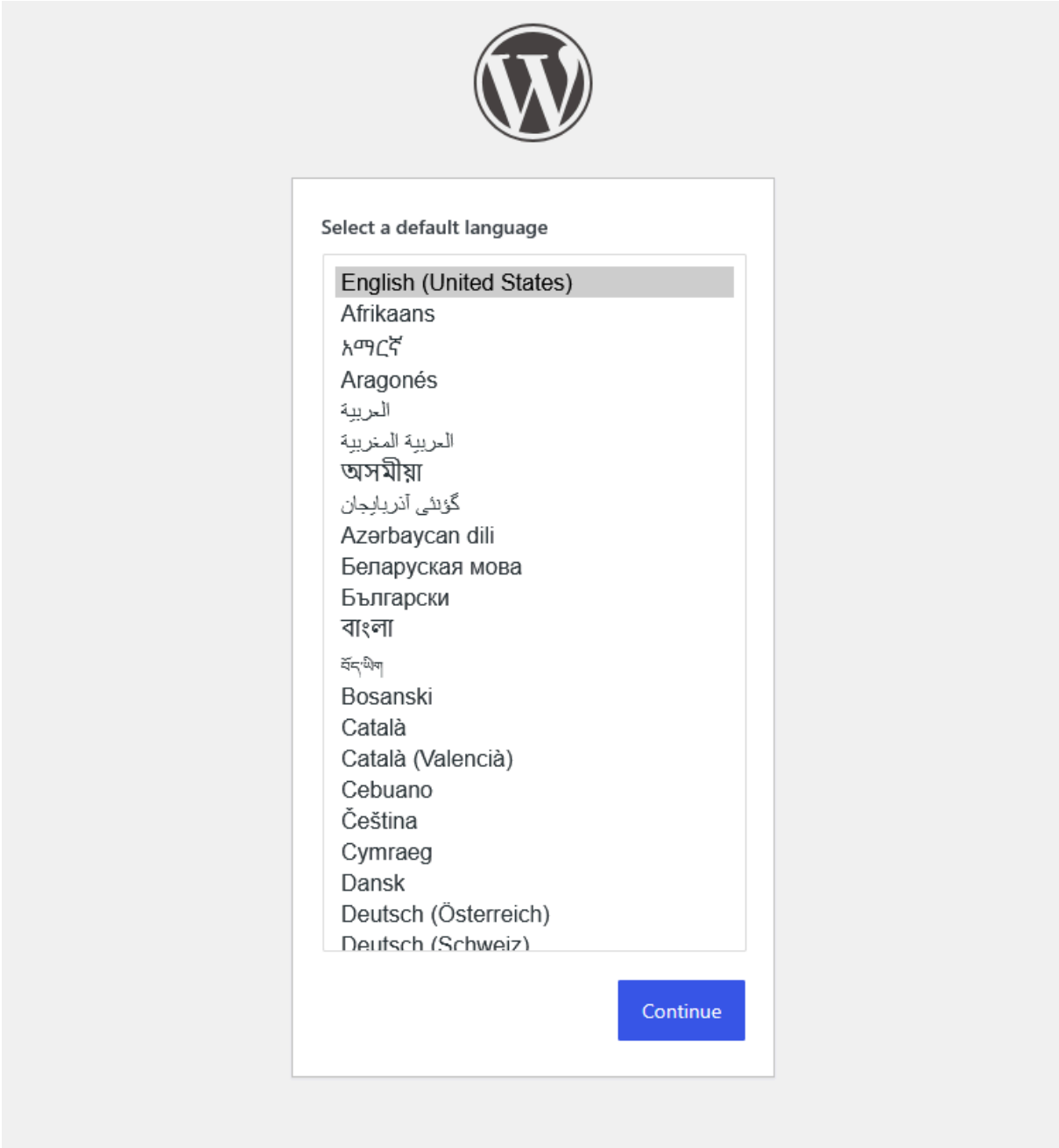
No resources found in database namespace.
No resources found in middle namespace.
```

Je redéploie dans les bons namespaces :

```
kubectl apply -f .\database\ -n database && kubectl apply -
f .\middle\ -n middle

configmap/mysql-configmap unchanged
secret/mysql-secret unchanged
service/mysql-svc unchanged
statefulset.apps/mysql-statefulset configured
namespace/database unchanged
namespace/middle unchanged
deployment.apps/wordpress-deployment created
```

```
persistentvolume/wordpress-pv unchanged
persistentvolumeclaim/wordpress-pvc created
service/wordpress-svc created
```



Parfait ! L'erreur venait du namespacing.

Vérifications

❓ Les différentes ressources sont déployées dans le bon namespace ? Quelle commande utilisez-vous pour vérifier cela ?

```
kubectl get pods -A

NAMESPACE     NAME                                      READY
STATUS    RESTARTS   AGE
database      mysql-statefulset-0                       1/1
Running   0          3m18s
default       mysql-statefulset-0                       1/1
Running   0          63m
default       wordpress-deployment-54fd46499c-d57pm     1/1
Running   0          25m
front         nginx-deployment-5d68b47d97-s8p4r         1/1
Running   0          3h41m
front         nginx-deployment-5d68b47d97-vs5bp         1/1
Running   0          3h41m
kube-system   coredns-559f6c778d-wtlgb                  1/1
Running   0          5h58m
kube-system   etcd-minikube                             1/1
Running   0          5h58m
kube-system   kindnet-nfd95                             1/1
Running   0          5h58m
kube-system   kube-apiserver-minikube                   1/1
Running   0          5h58m
kube-system   kube-controller-manager-minikube          1/1
Running   0          5h58m
kube-system   kube-proxy-qj67p                          1/1
Running   0          5h58m
kube-system   kube-scheduler-minikube                   1/1
Running   0          5h58m
kube-system   storage-provisioner                       1/1
Running   0          5h58m
middle        wordpress-deployment-54fd46499c-rgzn8     0/1
Pending   0          2m40s
```

> `kubectl get all -A` liste tous les ressources. `wordpress-deployment-...` est **PENDING**, car j'ai coupé la redirection du port entre temps.

❓ Les ressources MySQL ConfigMap et MySQL sont à déployer au sein des namespaces middle et database, pourquoi ?

Afin de séparer les responsabilités de chaque service (base de données, frontend, backend). Chaque namespace pourra recevoir des spécificités en fonction de ce qu'il contient, sans influencer d'autres services non concernés et améliorer la sécurité.

❓ Les différentes pods sont-ils correctement démarrés ? Quelle commande utilisez-vous pour vérifier cela ?

```
kubectl get pods -A
```

Le statut doit être **RUNNING** et la colonne **READY** indique **1/1** ou **2/2**, etc.

❓ Lorsqu'un pod est en erreur comment accédez-vous aux events du namespace et aux logs du pod ? Quelles commandes utilisez-vous pour vérifier cela ?

La première affichera les logs (l'option `-f` active le suivi des logs en direct), et la seconde affiche les événements d'un pod dans son namespace.

```
kubectl logs <pod> -n <namespace>
kubectl describe pod <pod> -n <namespace>
```